UNIFIL · CENTRO UNIVERSITÁRIO FILADÉLFIA



HYPE CARDS

Sistema web de controle de estoque e vendas para loja de Pokémon TCG

Documentação técnica do projeto: requisitos,
modelagem UML e banco de dados

Murillo Daigder

Londrina – PR · 2026

Sumário

1 Introdução

1.1 O sistema e o problema que resolve

1.2 Objetivos

1.3 Stack utilizada

1.4 Arquitetura

2 Desenvolvimento

2.1 Requisitos funcionais e não funcionais

2.2 Diagrama de casos de uso

2.3 Diagrama Entidade-Relacionamento (DER)

2.4 Diagrama de atividade

2.5 Diagrama de classes

2.6 Diagramas de sequência

3 Conclusão

4 Referências

A Apêndice A · Fluxo de autenticação

1. Introdução

O Hype Cards é um sistema web que controla o estoque e as vendas de uma loja de produtos Pokémon TCG (Trading Card Game). Ele registra cada venda, entrada e saída com data, valor e responsável, emite o recibo para o cliente e mostra o desempenho da loja em relatórios com gráficos. Tudo é acessado pelo navegador, sem planilhas.

1.1 O sistema e o problema que resolve

Uma loja de cartas colecionáveis vende três tipos de mercadoria com características bem diferentes:

- **Cartas avulsas**: cada uma tem coleção, número, raridade, condição de conservação e idioma, e uma única carta pode valer milhares de reais.
- **Produtos selados**: boosters, Elite Trainer Boxes, displays e latas, sempre ligados a uma coleção.
- **Acessórios**: sleeves, deck boxes, pastas e playmats, organizados por categoria e marca.

Na prática, o cliente leva vários itens de uma vez, pede desconto e paga por Pix, dinheiro ou cartão. Sem um sistema, o lojista não sabe com segurança quanto tem em estoque, quanto vendeu, qual foi o lucro, quem fez cada movimentação nem como desfazer uma venda errada. O Hype Cards resolve isso com vendas em carrinho, recibo, estorno, histórico completo, alerta de estoque baixo e relatórios por período.

O público é a própria loja: o **administrador** (dono ou gerente), que cadastra itens e usuários, estorna vendas e faz backup, e o **vendedor**, que registra vendas e movimentações e consulta o estoque e os relatórios.

1.2 Objetivos

- Centralizar o cadastro de coleções, cartas, produtos selados e acessórios, com foto de cada item.
- Registrar vendas com vários itens, desconto e forma de pagamento, emitindo o recibo para o cliente.
- Permitir o estorno de uma venda, devolvendo os itens ao estoque sem apagar o histórico.
- Registrar toda entrada e saída de estoque com motivo, valor e usuário, formando um histórico auditável.
- Mostrar faturamento, lucro, ticket médio e itens mais vendidos em relatórios por período, com gráficos.
- Avisar quando um item fica abaixo do estoque mínimo.

- Proteger o acesso com login e dois perfis de permissão, e permitir backup completo.

1.3 Stack utilizada

Figura 1 – Tecnologias utilizadas no Hype Cards



Fonte: elaborado pelo autor (2026), com logotipos do Simple Icons.

Quadro 1 – Camadas e tecnologias

CAMADA	TECNOLOGIA	USO NO PROJETO
Servidor	Node.js 22 ou superior	Executa a aplicação; módulos nativos `node:sqlite`, `node:crypto`, `node:zlib` e `node:test`
API	Express 5	Rotas REST, arquivos estáticos e tratamento de erros assíncronos
Banco principal	PostgreSQL 18 (driver `pg`)	Banco relacional usado em produção, com transações e trava de linha
Banco alternativo	SQLite (`node:sqlite`)	Banco em arquivo único, usado quando não há PostgreSQL configurado
Interface	HTML5, CSS3 e JavaScript em módulos	Telas, tema claro/escuro, gráficos em SVG desenhados sem bibliotecas e chamadas à API com `fetch`
Qualidade	node:test e GitHub Actions	25 testes automatizados da API, executados no SQLite e no PostgreSQL a cada push no GitHub

O projeto tem apenas duas dependências externas (`express` e `pg`). Senhas usam hash `scrypt`, o backup em ZIP é gerado com `zlib`, e a exportação CSV e os gráficos são feitos sem bibliotecas adicionais.

1.4 Arquitetura

O sistema segue a arquitetura cliente-servidor em três camadas: o navegador exibe as telas, a API Express aplica as regras de negócio e a camada de acesso a dados conversa com o banco. Essa última camada oferece a mesma interface para PostgreSQL e SQLite, e o banco é escolhido pela variável `DATABASE_URL` do arquivo `.env`. As fotos dos itens ficam em disco, na pasta `data/imagens`, e o banco guarda apenas o nome de cada arquivo.

No navegador, o código é dividido em 14 módulos JavaScript (`public/js`), um por responsabilidade: vendas, relatórios, gráficos, cadastros, movimentações, sessão e utilitários. No GitHub, um workflow de integração contínua roda todos os testes a cada envio de código.

2. Desenvolvimento

Esta seção descreve o que o sistema faz (requisitos) e como ele foi modelado: casos de uso, banco de dados, fluxo de atividades, classes e a troca de mensagens entre as partes em cada caso de uso.

2.1 Requisitos funcionais e não funcionais

O sistema tem 24 requisitos funcionais e 13 não funcionais. Na coluna Ator, "Todos" significa administrador e vendedor.

Quadro 2 – Requisitos funcionais

CÓDIGO	REQUISITO	ATOR
RF01	Autenticar o usuário com login e senha e encerrar a sessão (logout)	Todos
RF02	Exigir a troca da senha no primeiro acesso e permitir alterá-la depois	Todos
RF03	Gerenciar usuários: cadastrar, editar, desativar, redefinir senha e excluir, com perfil administrador ou vendedor	Administrador
RF04	Manter coleções (nome, sigla, série, data de lançamento)	Administrador
RF05	Manter cartas avulsas (coleção, número, raridade, condição, idioma, preço de venda, preço de custo, quantidade e estoque mínimo)	Administrador
RF06	Manter produtos selados (coleção, tipo, preços, quantidade e estoque mínimo)	Administrador
RF07	Manter acessórios (categoria, marca, preços, quantidade e estoque mínimo)	Administrador
RF08	Enviar e remover a foto de cada item, reduzindo a imagem automaticamente antes do envio	Administrador
RF09	Registrar entrada de estoque com motivo (Compra, Devolução de cliente, Ajuste de inventário) e custo pago	Todos
RF10	Registrar saída de estoque com motivo (Avaria/Perda, Uso interno, Ajuste de inventário ou venda rápida de um item), impedindo estoque negativo	Todos
RF11	Lançar automaticamente no histórico o estoque do cadastro inicial e toda alteração de quantidade feita pela edição	Sistema
RF12	Consultar o histórico geral com filtros (item, tipo, operação, motivo e período) e o histórico de cada item, com link para a venda de origem	Todos

CÓDIGO	REQUISITO	ATOR
RF13	Exibir o painel de resumo do mês: vendas e lucro, valor e custo do estoque, mais vendidos e itens abaixo do mínimo	Todos
RF14	Buscar por nome, filtrar por coleção e por estoque baixo, ordenar por coluna e paginar as listas	Todos
RF15	Alternar a listagem entre tabela e grade com fotos	Todos
RF16	Exportar listas e histórico para CSV, respeitando os filtros aplicados	Todos
RF17	Gerar backup em ZIP com o banco de dados e as imagens	Administrador
RF18	Alternar entre tema claro e escuro, lembrando a escolha	Todos
RF19	Registrar venda com vários itens (carrinho), com quantidade e preço por item, desconto em % ou R$ rateado entre os itens, forma de pagamento (Pix, dinheiro, débito ou crédito), cliente e observação	Todos
RF20	Recusar a venda inteira quando faltar saldo de qualquer item, informando quais itens faltam	Sistema
RF21	Emitir o recibo da venda para imprimir ou copiar e enviar ao cliente	Todos
RF22	Estornar uma venda, informando o motivo, devolvendo os itens ao estoque e mantendo a venda no histórico como estornada	Administrador
RF23	Listar e filtrar vendas por número, cliente, forma de pagamento, situação e período	Todos
RF24	Gerar relatórios por período, agrupados por mês ou por dia: faturamento, lucro, ticket médio, descontos, estornos, gráficos por categoria e por forma de pagamento e ranking dos itens	Todos

Quadro 3 – **Requisitos não funcionais**

CÓDIGO	CATEGORIA	REQUISITO
RNF01	Segurança	Senhas guardadas com hash `scrypt` e salt; sessão por cookie `HttpOnly` e `SameSite=Lax`, válida por 7 dias, com apenas o hash SHA-256 do token no banco
RNF02	Segurança	Bloquear o login por 1 minuto após 5 tentativas erradas do mesmo usuário e IP
RNF03	Segurança	Verificar o perfil em toda rota da API, não apenas esconder botões na tela

CÓDIGO	CATEGORIA	REQUISITO
RNF04	Integridade	Gravar venda, itens, saldo e movimentações na mesma transação, travando as linhas dos itens (PostgreSQL), e usar chaves estrangeiras e restrições `CHECK` (preço, total e quantidade ≥ 0)
RNF05	Portabilidade	Funcionar com PostgreSQL ou SQLite sem mudança de código, escolhendo o banco pela variável `DATABASE_URL`
RNF06	Desempenho	Paginar as listas (até 100 registros por página), indexar histórico e vendas por data e reduzir fotos para no máximo 1000 px em WEBP
RNF07	Usabilidade	Interface em português, responsiva até telas de celular, com diálogos próprios, prévia do saldo e total do carrinho calculado na hora
RNF08	Compatibilidade	Rodar em Node.js 22.13 ou superior e nos navegadores Chrome, Edge e Firefox atuais
RNF09	Manutenibilidade	Ter testes automatizados da API (25 cenários) executados nos dois bancos, no máximo duas dependências externas e o front-end dividido em módulos
RNF10	Rastreabilidade	Manter o histórico legível mesmo após excluir o item ou o usuário, copiando seus nomes para a venda e para a movimentação
RNF11	Validação	Validar todos os dados no servidor com mensagens claras; aceitar imagens PNG, JPG, WEBP ou GIF de até 5 MB
RNF12	Interoperabilidade	Gerar CSV que abre direto no Excel em português: separador `;`, vírgula decimal e codificação UTF-8 com BOM
RNF13	Qualidade	Executar os testes automaticamente no GitHub Actions a cada push, em Linux, no SQLite e em um PostgreSQL de teste

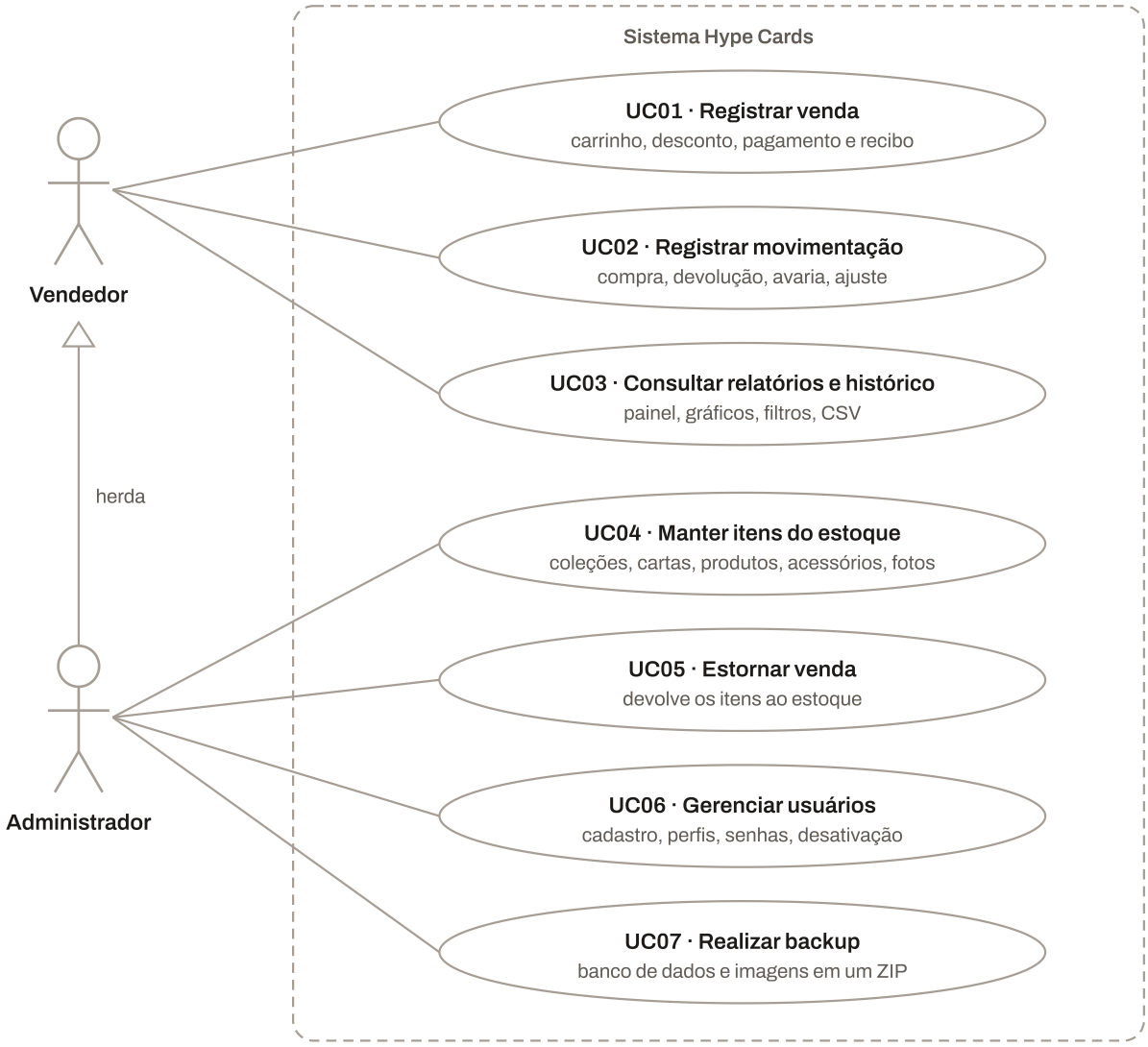
2.2 Diagrama de casos de uso

O sistema tem dois atores e sete casos de uso. O vendedor registra vendas e movimentações e consulta os relatórios. O administrador herda tudo o que o vendedor faz (generalização) e, além disso, mantém o cadastro de itens, estorna vendas, gerencia usuários e realiza o backup.

Por que o login não é um caso de uso. O login não entrega um resultado de negócio por si só: ele é a **pré-condição** de todos os casos de uso. Por isso aparece nos requisitos (RF01 e RF02), na segurança (RNF01 e RNF02) e no Apêndice A, que mostra o fluxo de autenticação.

Figura 2 – Diagrama de casos de uso · 2 atores, 7 casos de uso

O vendedor vende e movimenta o estoque; o administrador faz todo o resto



Fonte: elaborado pelo autor (2026).

Quadro 4 – Descrição dos casos de uso

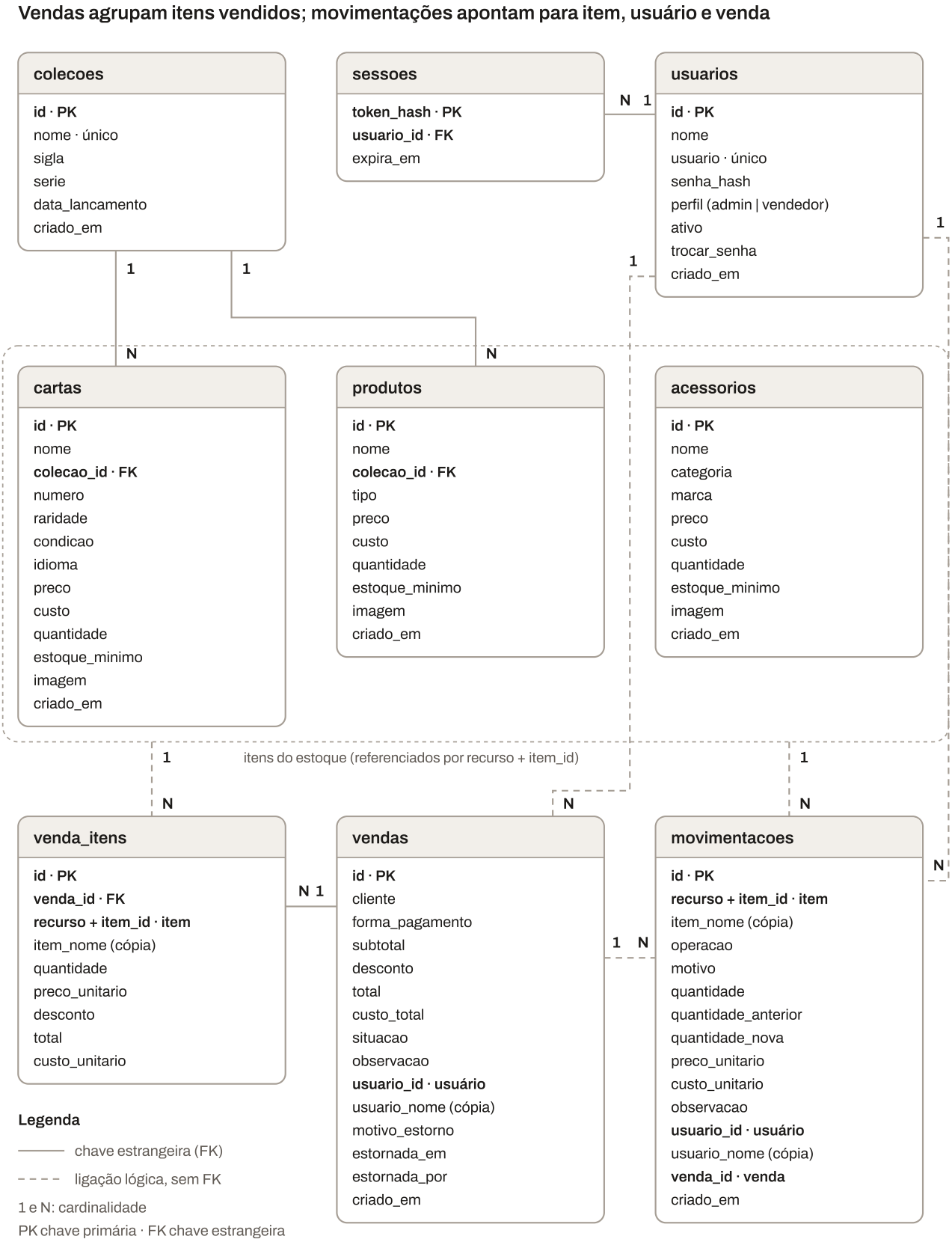
CASO DE USO	ATORES	DESCRIÇÃO	PRÉ-CONDIÇÃO	RESULTADO
UC01 · Registrar venda	Vendedor, Administrador	Monta o carrinho com um ou mais itens, aplica desconto, escolhe a forma de pagamento e finaliza; o recibo é exibido para imprimir ou copiar	Autenticado; itens com saldo	Venda gravada, estoque baixado e movimentações ligadas à venda

CASO DE USO	ATORES	DESCRIÇÃO	PRÉ-CONDIÇÃO	RESULTADO
UC02 · Registrar movimentação	Vendedor, Administrador	Registra entrada (compra, devolução, ajuste) ou saída (avaria, uso interno, ajuste) de um item, com quantidade e valor	Autenticado; item cadastrado	Saldo atualizado e movimentação gravada no histórico
UC03 · Consultar relatórios e histórico	Vendedor, Administrador	Consulta o painel do mês, os relatórios por período com gráficos, a lista de vendas e o histórico; exporta em CSV	Autenticado	Informação exibida ou arquivo CSV baixado
UC04 · Manter itens do estoque	Administrador	Cadastra, edita e exclui coleções, cartas, produtos e acessórios, com foto opcional	Autenticado como administrador	Item salvo; quantidade inicial lançada no histórico
UC05 · Estornar venda	Administrador	Informa o motivo e estorna uma venda concluída; os itens voltam para o estoque	Autenticado como administrador; venda concluída	Venda marcada como estornada e movimentações de estorno gravadas
UC06 · Gerenciar usuários	Administrador	Cadastra, edita, desativa, redefine a senha e exclui usuários, definindo o perfil de cada um	Autenticado como administrador	Usuário salvo; no primeiro acesso, ele define a própria senha
UC07 · Realizar backup	Administrador	Gera um arquivo ZIP com a cópia do banco de dados e todas as fotos dos itens	Autenticado como administrador	Arquivo de backup baixado

2.3 Diagrama Entidade-Relacionamento (DER)

O banco tem nove tabelas. Cartas e produtos pertencem a uma coleção por chave estrangeira. Uma venda (`vendas`) é composta pelos itens vendidos (`venda_itens`), e cada item vendido gera uma movimentação ligada à venda. Vendas, itens vendidos e movimentações guardam uma cópia do nome do item e do usuário, para continuarem legíveis depois de exclusões.

Figura 3 – Diagrama Entidade-Relacionamento · 9 tabelas



Fonte: elaborado pelo autor (2026).

Quadro 5 – Dicionário de dados resumido

TABELA	GUARDA	CHAVE PRIMÁRIA	RELACIONAMENTOS	REGRAS PRINCIPAIS
`colecoes`	Coleções de Pokémon TCG	id	1:N com cartas e com produtos	`nome` único; não pode ser excluída com itens vinculados (`ON DELETE RESTRICT`)
`cartas`	Cartas avulsas	id	N:1 com colecoes; 1:N com venda_itens e movimentacoes	`preco` , `custo` , `quantidade` e `estoque_minimo` ≥ 0 (`CHECK`)
`produtos`	Produtos selados	id	N:1 com colecoes; 1:N com venda_itens e movimentacoes	Mesmas regras de estoque das cartas; `tipo` obrigatório
`acessorios`	Acessórios	id	1:N com venda_itens e movimentacoes	Mesmas regras de estoque; `categoria` obrigatória
`vendas`	Cabeçalho de cada venda	id (número da venda)	1:N com venda_itens e movimentacoes; N:1 com usuarios	`situacao` ∈ {concluida, estornada}; `total` = `subtotal` – `desconto` ; estorno registra motivo, data e responsável
`venda_itens`	Itens de cada venda	id	N:1 com vendas (`ON DELETE CASCADE`); N:1 com o item (`recurso` + `item_id`)	`quantidade` > 0; desconto rateado por item; custo do item na data da venda
`movimentacoes`	Histórico de entradas e saídas	id	N:1 com o item, com usuarios e, nas vendas e estornos, com vendas (`venda_id`)	`operacao` ∈ {entrada, saida}; `quantidade` > 0; índices por item, data e venda
`usuarios`	Contas de acesso	id	1:N com sessoes, vendas e movimentacoes	Login único sem diferenciar maiúsculas; `perfil` ∈ {admin, vendedor}; senha só como hash

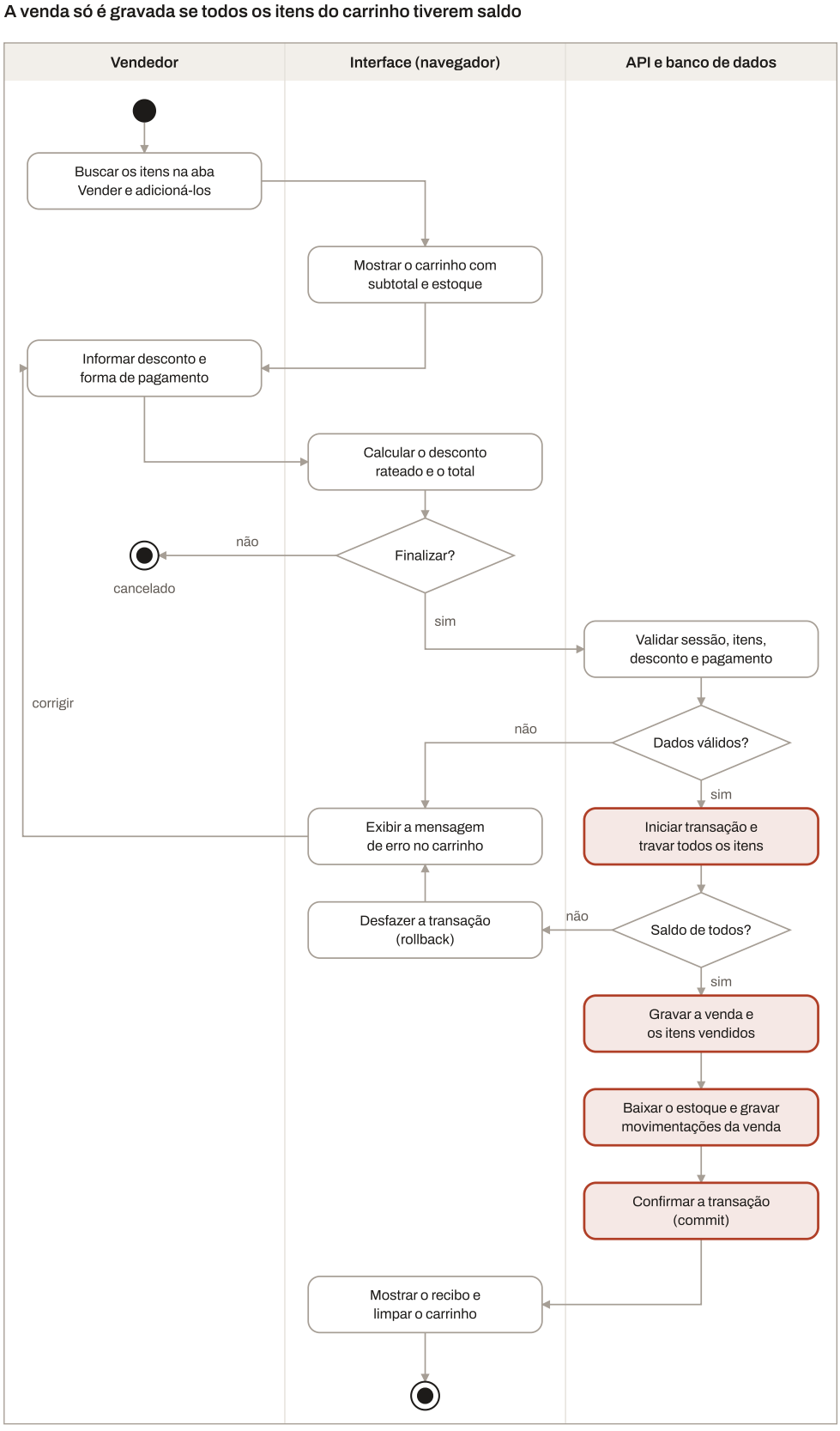
TABELA	GUARDA	CHAVE PRIMÁRIA	RELACIONAMENTOS	REGRAS PRINCIPAIS
`sessoes`	Logins ativos	token_hash	N:1 com usuarios	Excluídas junto com o usuário (`ON DELETE CASCADE`); expiram em 7 dias

Os campos `criado_em` recebem a data e hora do servidor automaticamente. No PostgreSQL os ids são `IDENTITY` e os valores monetários `DOUBLE PRECISION` ; no SQLite, `AUTOINCREMENT` e `REAL` . Vendas registradas antes da criação da tabela `vendas` são convertidas automaticamente na inicialização, com forma de pagamento “Não informada”.

2.4 Diagrama de atividade

O diagrama mostra o fluxo mais importante do sistema, o registro de uma venda com carrinho (caso de uso UC01). Todos os itens são travados e gravados numa única transação (passos destacados): se faltar saldo de qualquer item, nada é gravado.

Figura 4 – Diagrama de atividade · registrar uma venda · 3 raias, 3 decisões



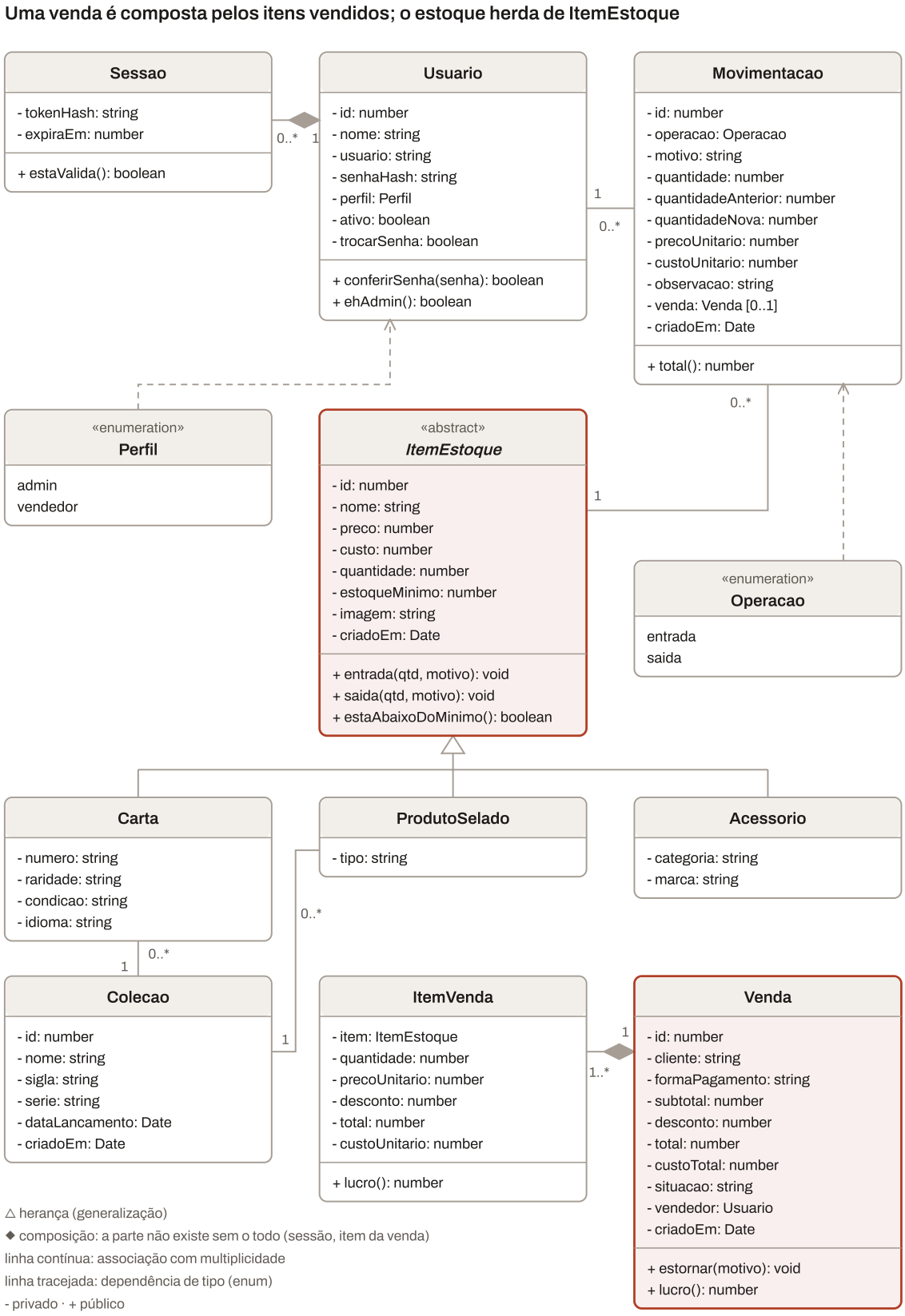
Fonte: elaborado pelo autor (2026).

O desconto informado em % ou em R$ é repartido entre os itens na proporção do valor de cada um, e o último item recebe a diferença de arredondamento. Assim, a soma dos itens bate com o total da venda e o lucro por item continua correto. A trava das linhas impede que duas vendas simultâneas leiam o mesmo saldo e deixem o estoque negativo.

2.5 Diagrama de classes

O diagrama representa o modelo conceitual das entidades. A classe abstrata `ItemEstoque` concentra o que cartas, produtos selados e acessórios têm em comum. A classe `Venda` é composta por um ou mais `ItemVenda`, cada um apontando para um item do estoque.

Figura 5 – Diagrama de classes · entidades do domínio · 12 classes



Fonte: elaborado pelo autor (2026).

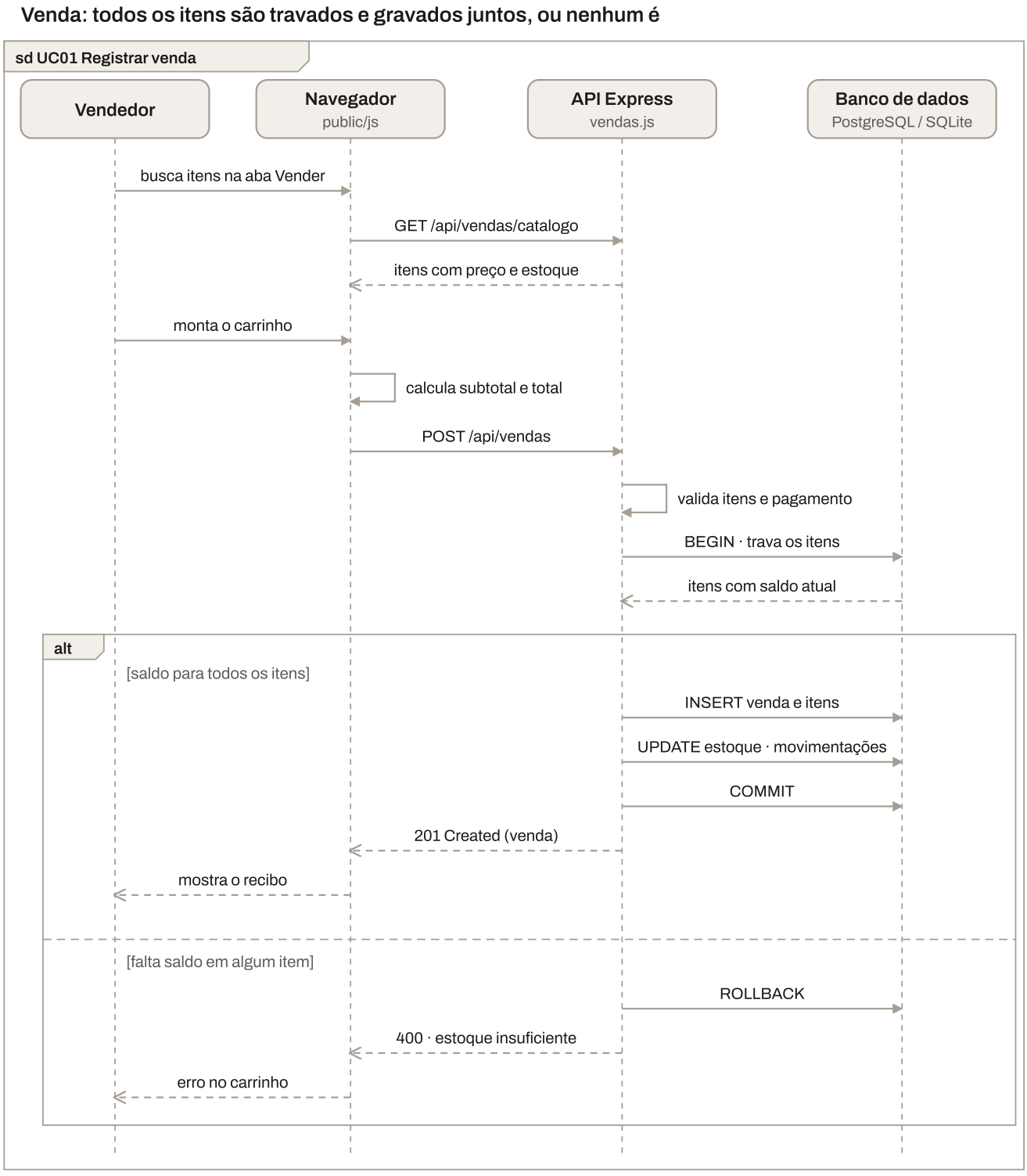
No código, escrito em JavaScript sem orientação a objetos formal, essa herança aparece como um gerador genérico de rotas (`crud.js`): os campos comuns são declarados uma vez (`camposEstoque` em `recursos.js`) e reaproveitados pelas três tabelas. Os métodos do diagrama correspondem a rotas da API, por exemplo `entrada()` e `saida()` → `PATCH /api/{recurso}/{id}/estoque` e `estornar()` → `POST /api/vendas/{id}/estorno` .

2.6 Diagramas de sequência

Cada caso de uso tem seu diagrama de sequência, com quatro participantes: o usuário, o navegador (módulos em `public/js`), a API Express com o módulo responsável e o banco de dados. Setas cheias são chamadas; setas tracejadas, respostas. Os quadros `alt` mostram caminhos alternativos. Todos partem de um usuário já autenticado; o fluxo de autenticação está no Apêndice A.

UC01 · Registrar venda

Figura 6 – Sequência UC01 · venda com carrinho

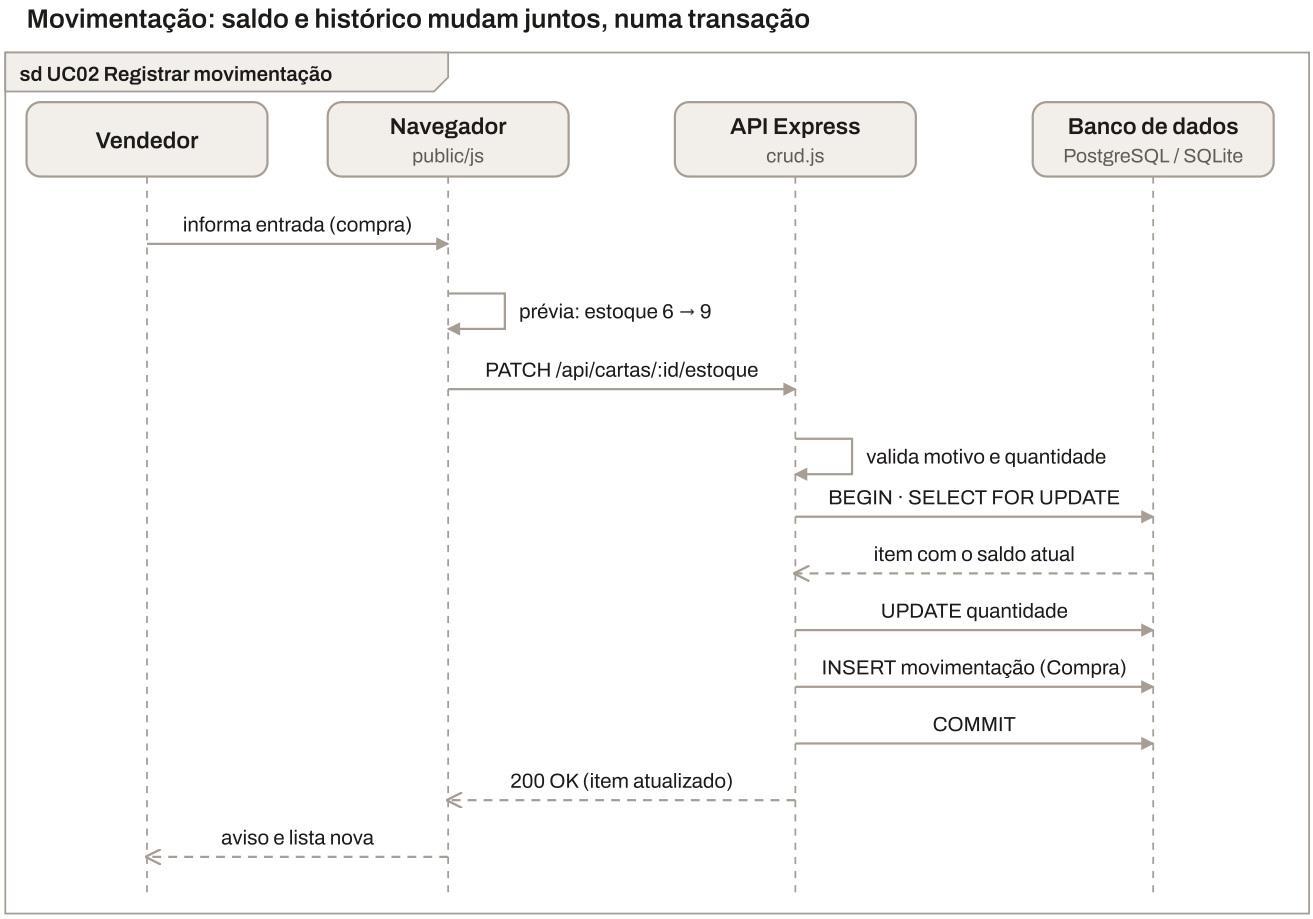


Fonte: elaborado pelo autor (2026).

A API trava os itens em ordem fixa (por tipo e id), o que evita impasse entre duas vendas com os mesmos itens. No PostgreSQL a trava é o `SELECT ... FOR UPDATE`; no SQLite, a camada de acesso a dados enfileira as transações com o mesmo efeito.

UC02 · Registrar movimentação

Figura 7 – Sequência UC02 · entrada de estoque por compra

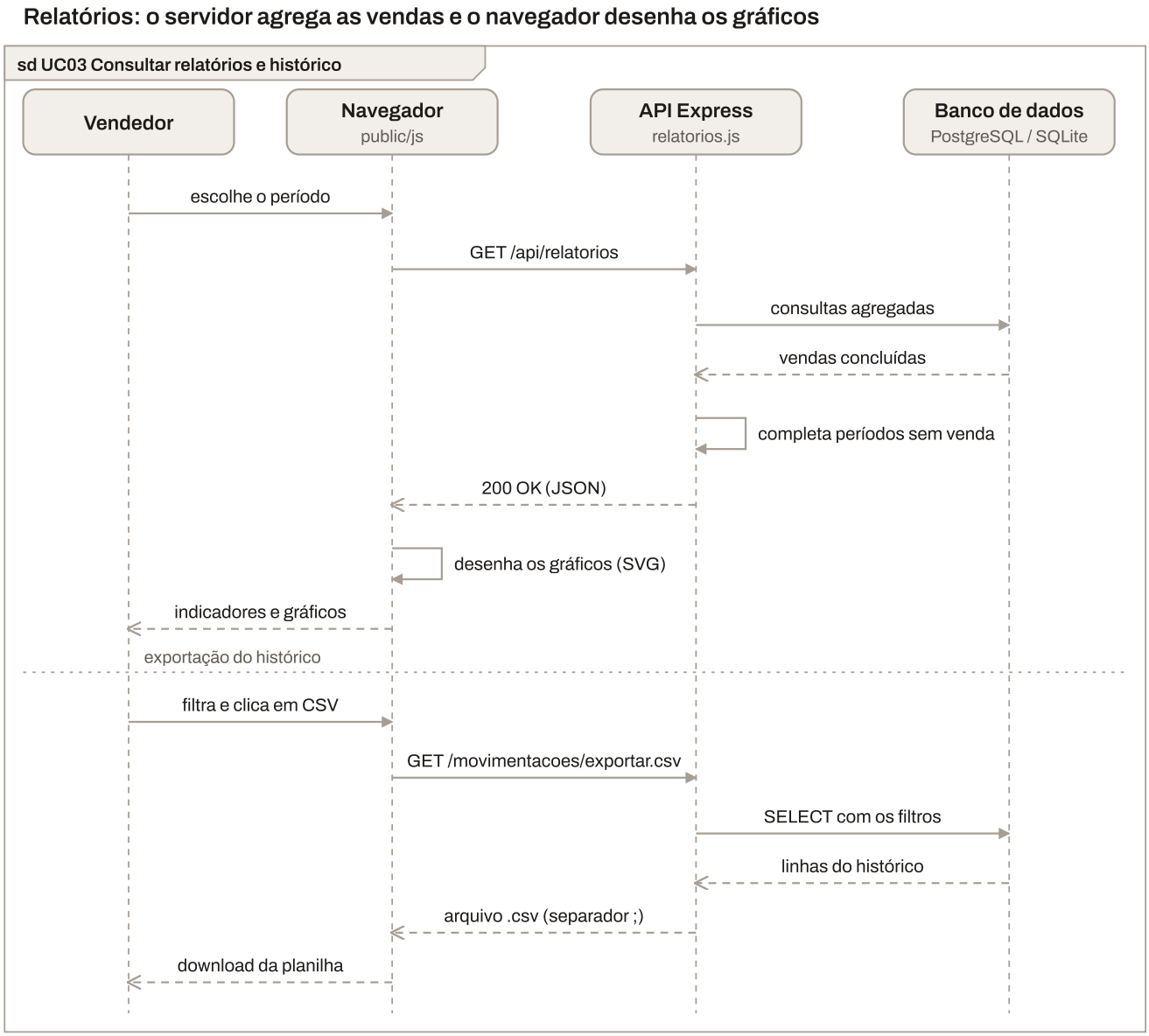


Fonte: elaborado pelo autor (2026).

Uma saída maior que o saldo é recusada com a resposta 400 e nada é gravado. Uma saída com motivo “Venda” feita por este caminho segue o fluxo do UC01, como uma venda de um item só.

UC03 · Consultar relatórios e histórico

Figura 8 – Sequência UC03 · relatório por período e exportação CSV

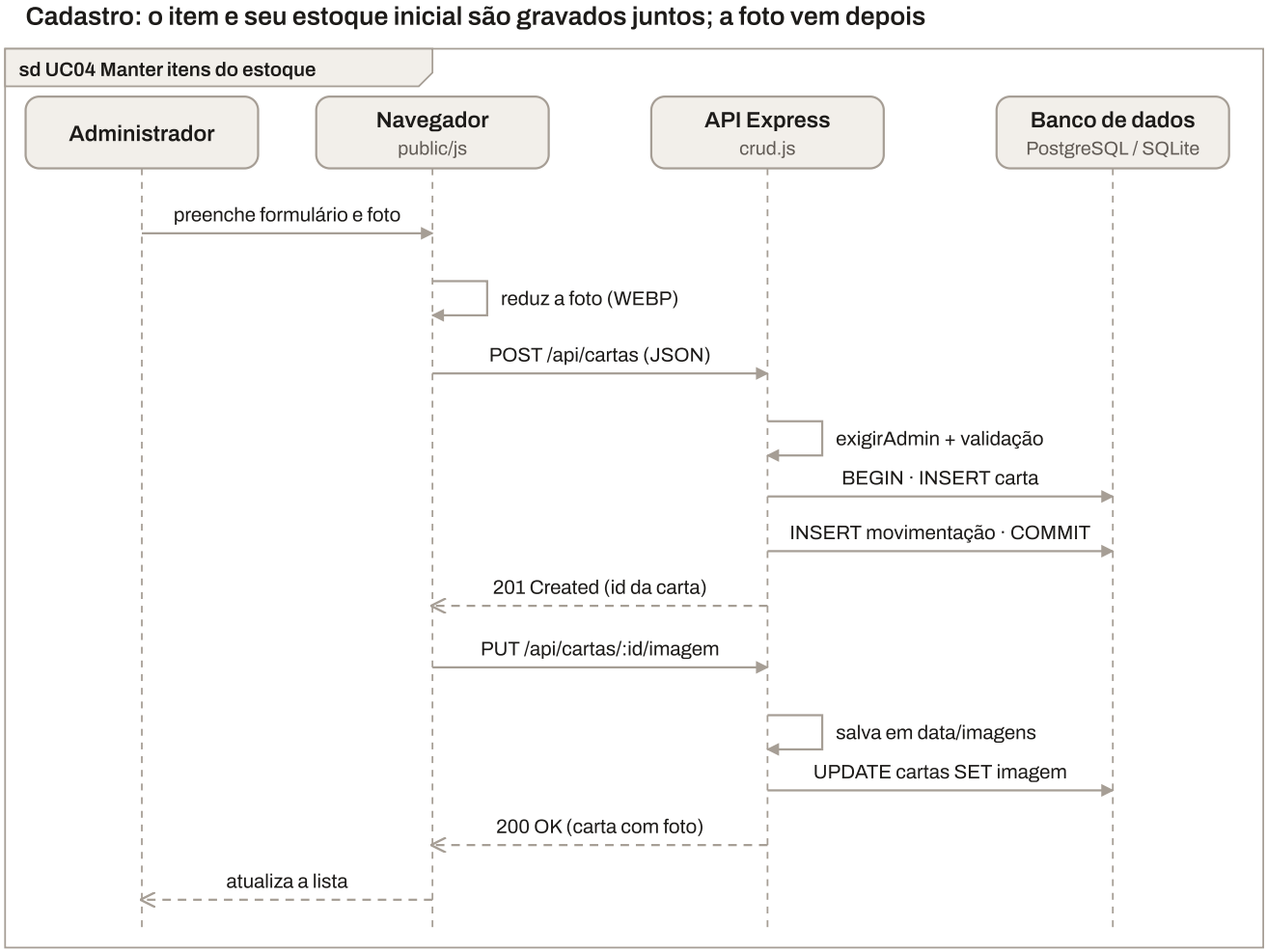


Fonte: elaborado pelo autor (2026).

O relatório considera só vendas concluídas: as estornadas aparecem separadas, no indicador de estornos. O servidor completa com zero os meses ou dias sem venda, para o gráfico mostrar o período inteiro.

UC04 · Manter itens do estoque

Figura 9 – Sequência UC04 · cadastro de carta com foto

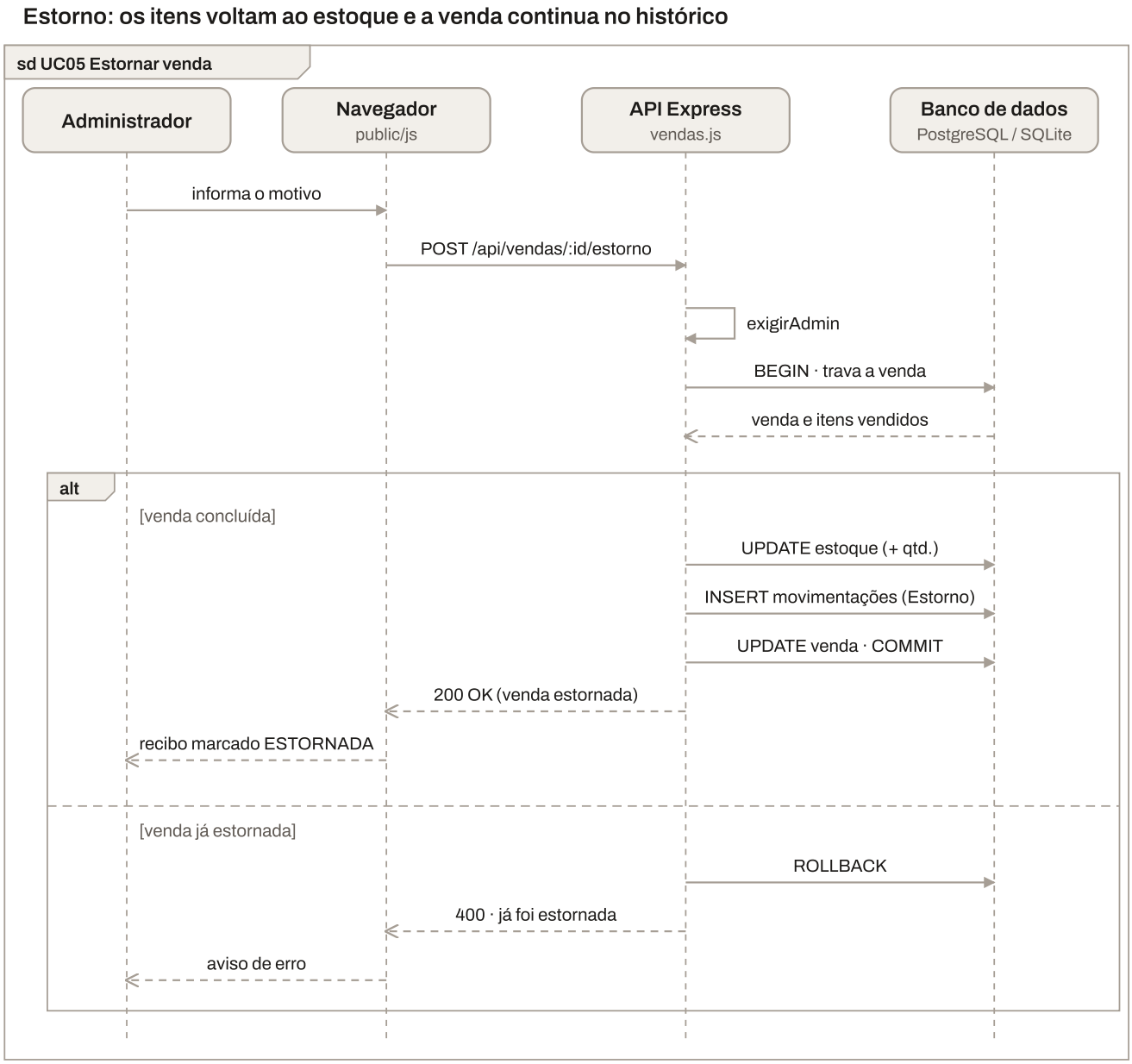


Fonte: elaborado pelo autor (2026).

O exemplo é o cadastro de uma carta. A edição (`PUT`) e a exclusão (`DELETE`) seguem o mesmo caminho; se a edição mudar a quantidade, a diferença entra no histórico como “Ajuste de inventário”.

UC05 · Estornar venda

Figura 10 – Sequência UC05 · estorno de uma venda

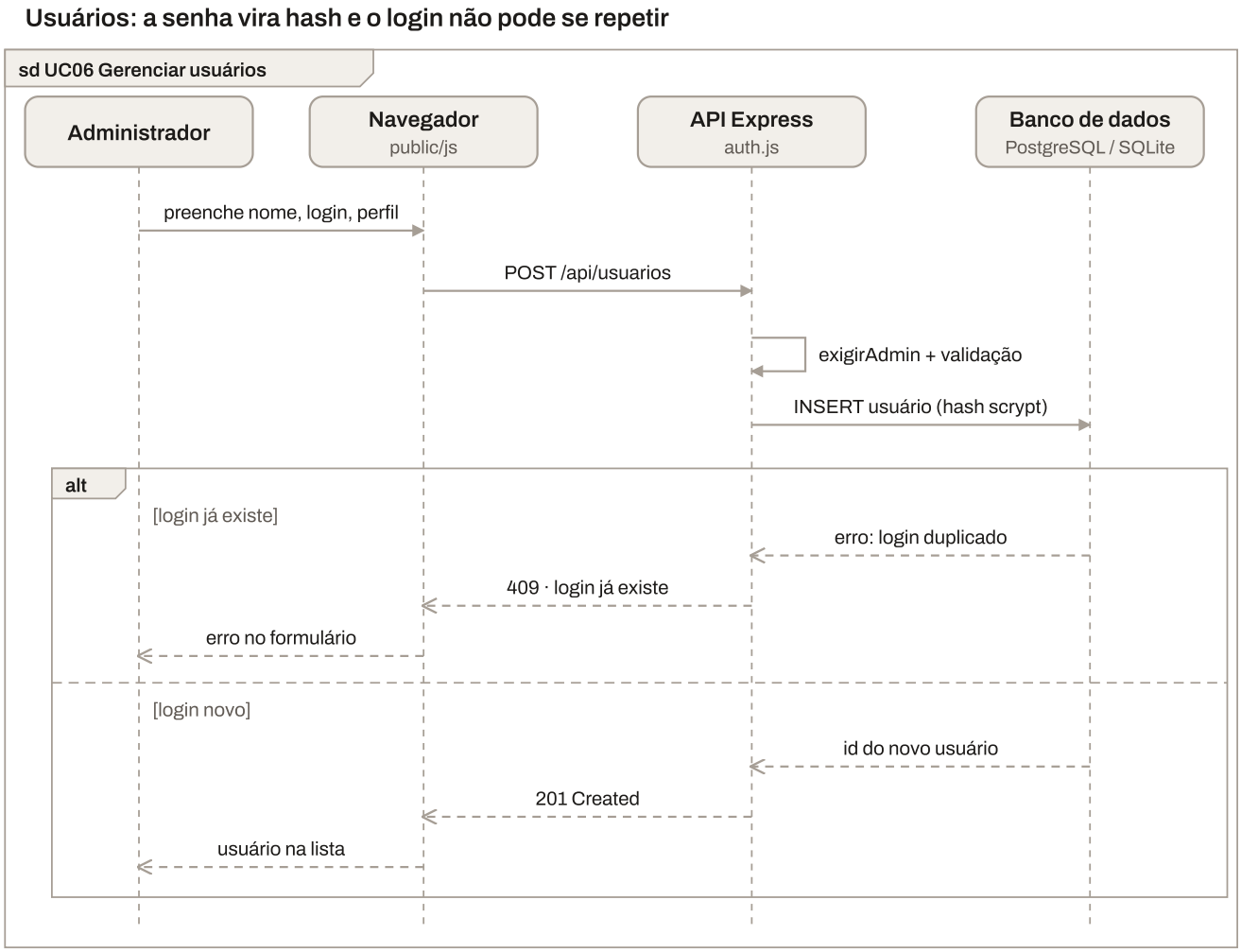


Fonte: elaborado pelo autor (2026).

O estorno não apaga nada: a venda continua no histórico com a situação “estornada”, o motivo, a data e quem estornou, e cada item devolvido gera uma movimentação “Estorno de venda”. Um vendedor que chame essa rota recebe a resposta 403.

UC06 · Gerenciar usuários

Figura 11 – Sequência UC06 · cadastro de um usuário

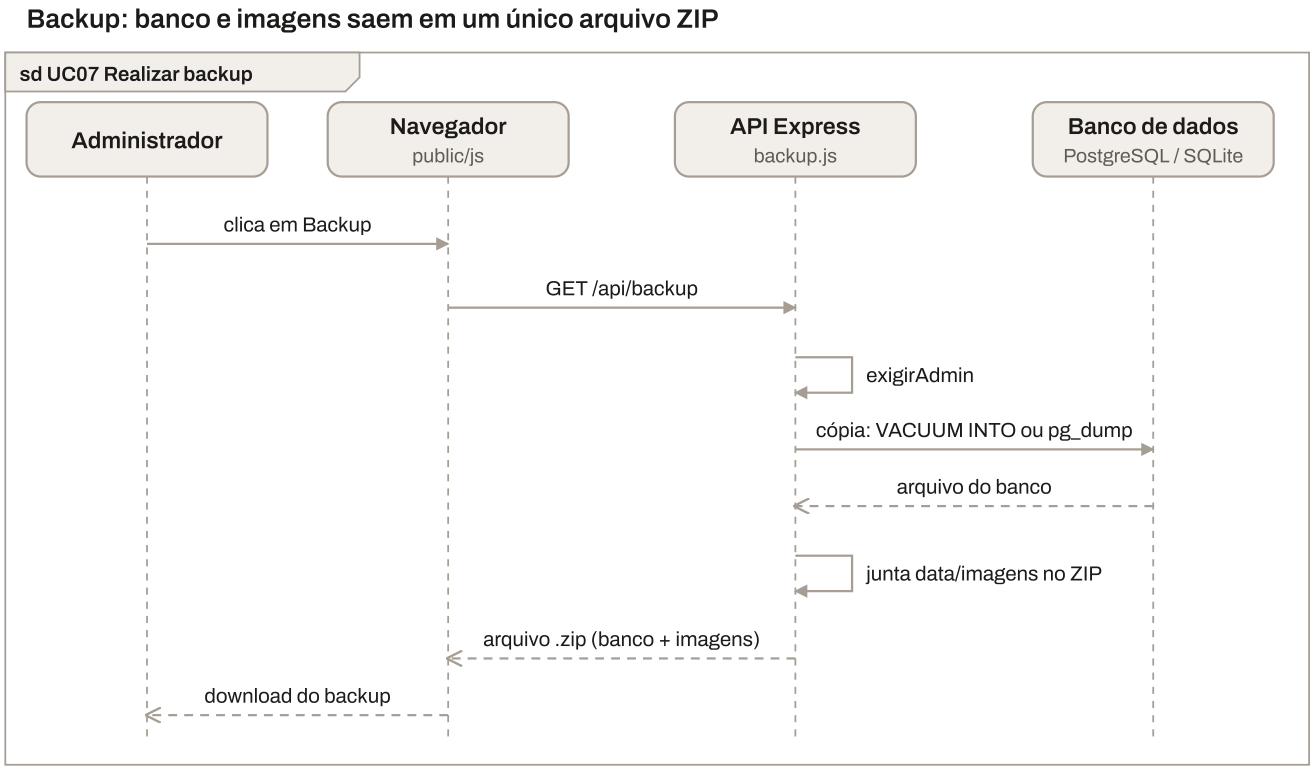


Fonte: elaborado pelo autor (2026).

O usuário criado recebe `trocar_senha = 1` e precisa definir a própria senha no primeiro acesso. A API impede excluir o próprio usuário e remover o último administrador ativo; desativar um usuário ou redefinir sua senha encerra as sessões abertas dele.

UC07 · Realizar backup

Figura 12 – Sequência UC07 · geração do backup



Fonte: elaborado pelo autor (2026).

No SQLite, a cópia do banco usa `VACUUM INTO`, que gera um arquivo consistente mesmo com o sistema em uso; no PostgreSQL, o `pg_dump` gera um script SQL. Um vendedor que chame essa rota recebe a resposta 403.

3. Conclusão

O Hype Cards atingiu os objetivos propostos: controla o estoque das três categorias da loja, registra vendas com carrinho, desconto, recibo e estorno, guarda cada movimentação com responsável e valor e mostra faturamento e lucro em relatórios com gráficos. O sistema roda em PostgreSQL ou SQLite, e os 25 testes automatizados passam nos dois bancos, também no GitHub a cada envio de código.

O que aprendi com o projeto:

- **Modelagem antes do código.** Separar cartas, produtos e acessórios, mas com os mesmos campos de estoque, permitiu um único módulo genérico de cadastro. Separar a venda (cabeçalho) dos itens vendidos deixou o histórico agrupado e o estorno simples.
- **Transações e concorrência.** Uma venda com vários itens precisa ser tudo ou nada. Travar todos os itens, sempre na mesma ordem, evita estoque negativo e impasse entre duas vendas ao mesmo tempo.
- **Dinheiro exige cuidado.** Repartir um desconto entre itens gera diferenças de centavos; foi preciso arredondar cada parte e jogar o resto no último item para a soma bater com o total.
- **Segurança na prática.** Senha nunca é guardada em texto puro, o token de sessão também não, e a permissão precisa ser conferida no servidor. Esconder um botão na tela não protege nada.
- **Bancos diferentes, SQL diferente.** A migração do SQLite para o PostgreSQL mostrou diferenças reais: marcadores `?` contra `$1`, `AUTOINCREMENT` contra `IDENTITY`, `LIKE` contra `ILIKE`, funções de data e códigos de erro próprios. Isolar o acesso a dados numa camada única deixou a troca restrita a poucos arquivos.
- **Testes e integração contínua valem o esforço.** Rodar os mesmos testes nos dois bancos encontrou erros que só apareciam no PostgreSQL, e o GitHub Actions passou a conferir tudo sozinho a cada envio.
- **Organização do front-end.** Dividir um arquivo de cerca de 860 linhas em 14 módulos tornou cada tela mais fácil de entender e de mudar sem quebrar as outras.

Como evolução, o sistema pode ganhar cadastro de clientes e fornecedores, emissão de nota fiscal, integração com uma loja virtual e publicação em um servidor na internet.

4. Referências

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

EXPRESS. **Express 5.x: API reference**. Disponível em: https://expressjs.com/en/5x/api.html. Acesso em: 14 out. 2026.

GITHUB. **GitHub Actions documentation**. Disponível em: https://docs.github.com/actions. Acesso em: 19 out. 2026.

GUEDES, Gilleanes T. A. **UML 2: uma abordagem prática**. 3. ed. São Paulo: Novatec, 2018.

HEUSER, Carlos Alberto. **Projeto de banco de dados**. 6. ed. Porto Alegre: Bookman, 2009.

MDN WEB DOCS. **HTML, CSS e JavaScript**. Mozilla. Disponível em: https://developer.mozilla.org/pt-BR/. Acesso em: 14 out. 2026.

NODE-POSTGRES. **node-postgres documentation**. Disponível em: https://node-postgres.com/. Acesso em: 14 out. 2026.

NODE.JS. **Node.js v22 documentation**. OpenJS Foundation. Disponível em: https://nodejs.org/docs/latest-v22.x/api/. Acesso em: 14 out. 2026.

OBJECT MANAGEMENT GROUP. **OMG Unified Modeling Language (UML), version 2.5.1**. 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1/. Acesso em: 14 out. 2026.

OWASP. **Password storage cheat sheet**. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Acesso em: 14 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 18 documentation**. Disponível em: https://www.postgresql.org/docs/18/. Acesso em: 14 out. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.

SIMPLE ICONS. **Simple Icons: free SVG icons for popular brands**. Disponível em: https://simpleicons.org/. Acesso em: 14 out. 2026.

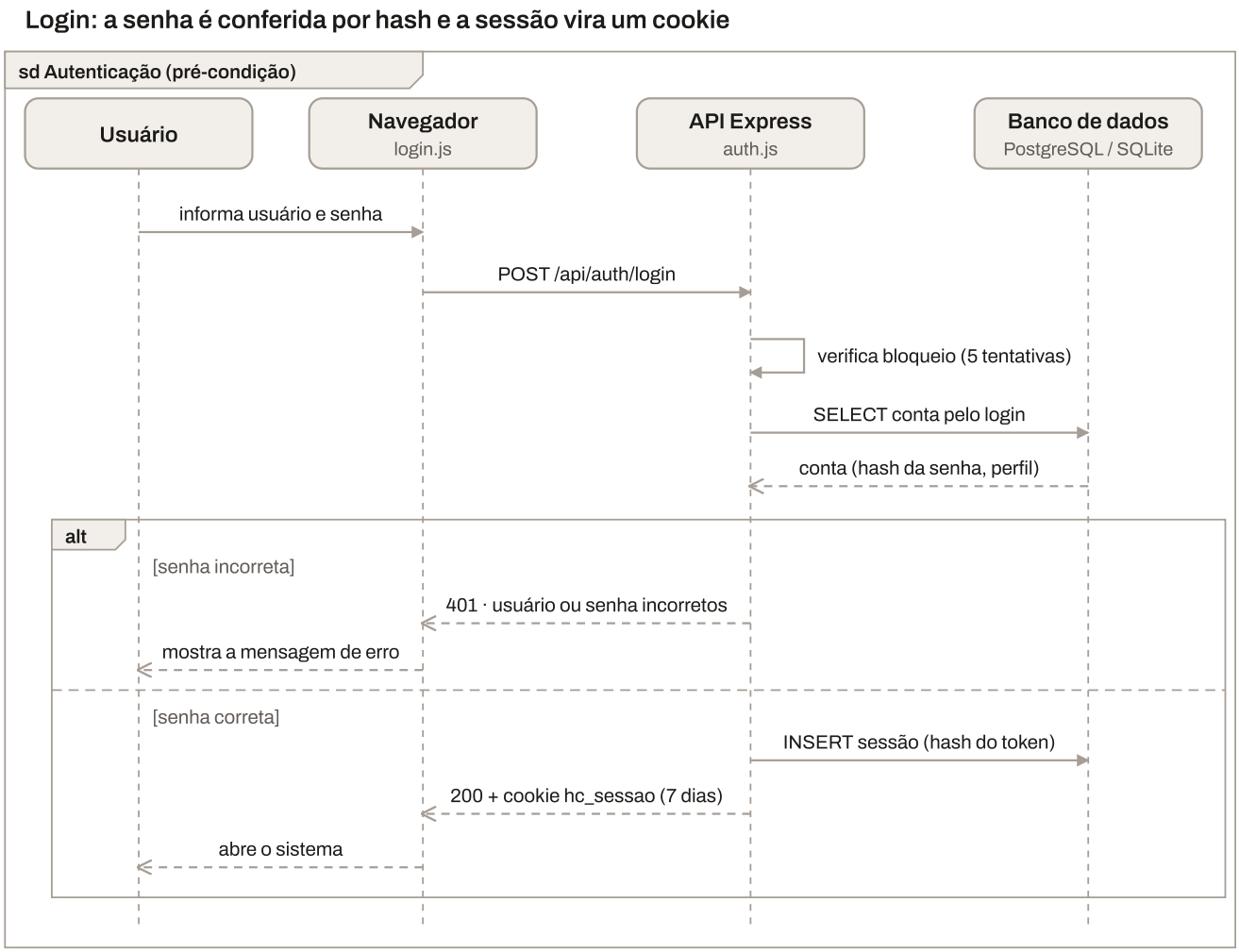
SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2019.

SQLITE. **SQLite documentation**. Disponível em: https://www.sqlite.org/docs.html. Acesso em: 14 out. 2026.

Apêndice A · Fluxo de autenticação

A autenticação é a pré-condição de todos os casos de uso. O diagrama mostra o login: o servidor confere a senha pelo hash `scrypt`, bloqueia o acesso por 1 minuto após 5 tentativas erradas e, quando a senha está correta, abre uma sessão de 7 dias guardada em cookie. No banco fica apenas o hash SHA-256 do token da sessão.

Figura 13 – Sequência da autenticação (pré-condição dos casos de uso)



Fonte: elaborado pelo autor (2026).